

DART XML 원문 구조 조사

항목	내용
작업 단계	04
작업 영역	공시 원문 구조 조사
조사 대상	disclosure_documents에 등록된 DART_XML 문서
조사 수량	3,147개
조사 완료일	2026-08-10
최종 상태	완료 (SUCCESS 3,147건, FAILURE 0건)

1. 작업 목적

대회에서 제공한 DART XML 원문은 파일마다 길이와 공시 내용이 다르고, 일부 파일에는 XML 문법에 맞지 않는 문자가 포함되어 있다.

원문 파서를 바로 구현하면 다음 문제를 뒤늦게 발견할 수 있다.

- 어떤 태그가 장·절·제목·문단·표를 나타내는지 알기 어렵다.
- 파일 하나가 매우 커서 전체 내용을 메모리에 올리는 방식이 위험할 수 있다.
- XML 문법 오류가 있는 파일에서 전체 배치가 중단될 수 있다.
- 본문에 사용된 <문자열>을 실제 XML 태그로 잘못 해석할 수 있다.
- 정상 파일 몇 개만 보고 만든 파서가 다른 공시 유형에서 실패할 수 있다.

따라서 실제 본문을 DB에 저장하기 전에 전체 DART XML 파일을 대상으로 구조와 오류 유형을 조사했다.

이 작업의 목적은 공시 내용을 분석하는 것이 아니라, **향후 원문 파서가 안전하게 읽어야 할 XML 구조와 예외 사례를 확인하는 것이다.**

2. 조사 범위와 완료 기준

조사 범위

- DB의 disclosure_documents 테이블에 등록된 문서
- content_format = DART_XML인 문서만 대상
- document_role이 MAIN 또는 ATTACHMENT인 XML 파일
- 페이지당 최대 500개씩 총 7페이지 조사

HTML과 PDF는 이번 조사 범위에 포함하지 않았다.

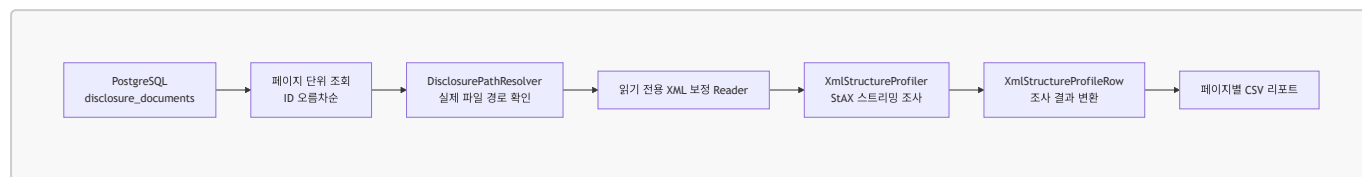
완료 기준

- DB에 등록된 DART XML 3,147개가 모두 결과 CSV에 존재한다.
- 동일한 disclosure_document_id가 중복 기록되지 않는다.
- 모든 페이지의 ID 범위가 앞 페이지 이후부터 이어진다.
- 각 파일의 조사 성공 또는 실패와 실패 원인이 기록된다.

- 최종 실패 건수가 0건이다.

최종 검증 결과 위 조건을 모두 충족했다.

3. 전체 처리 흐름



1. DB에서 DART XML 문서를 ID 오름차순으로 조회한다.
2. `relative_path`를 실제 데이터셋 파일 경로로 변환한다.
3. 원본 파일을 변경하지 않고, 읽는 동안 XML 문법 오류를 임시 보정한다.
4. StAX 방식으로 파일을 앞에서부터 한 번씩 읽으며 태그 수와 깊이를 집계한다.
5. 성공 결과 또는 최하위 예외 정보를 `XmlStructureProfileRow`로 만든다.
6. 각 페이지 결과를 UTF-8 BOM이 포함된 CSV로 저장한다.

4. 주요 구현 구성요소

구성요소	역할
<code>XmlStructureProfile</code>	XML 파일 하나의 구조 조사 결과와 태그별 개수를 보관한다.
<code>XmlStructureProfiler</code>	XML을 스트리밍 방식으로 읽고 루트, 깊이, 태그 수 등을 집계한다.
<code>XmlStructureProfileStatus</code>	파일별 조사 상태를 <code>SUCCESS</code> , <code>FAILURE</code> 로 구분한다.
<code>XmlStructureProfileRow</code>	공시 메타데이터, 구조 조사 결과, 오류 정보를 CSV 한 행 형태로 표현한다.
<code>XmlStructureProfileBatchResult</code>	한 페이지의 전체 행과 성공-실패 개수를 보관한다.
<code>XmlStructureProfileBatchService</code>	DB 페이지 조회, 경로 해석, 파일별 조사, 실패 격리를 담당한다.
<code>XmlStructureProfileReportWriter</code>	배치 결과를 CSV 파일로 저장한다.
<code>XmlStructureProfileBatchRunner</code>	환경 설정에 따라 애플리케이션 시작 시 지정 페이지를 조사한다.
<code>DisclosureDocumentRepository</code>	조사할 DART XML 문서를 ID 오름차순으로 페이지 조회한다.
<code>DisclosurePathResolver</code>	DB의 상대 경로를 실제 원문 파일 경로로 안전하게 변환한다.

5. 수집한 결과 항목

공시 및 파일 식별 정보

CSV 필드	의미
<code>disclosure_document_id</code>	<code>disclosure_documents</code> 기본 키
<code>source_doc_id</code>	데이터셋에서 제공한 원문 식별자
<code>receipt_no</code>	DART 접수번호

CSV 필드	의미
source_group	공시 그룹 (major , periodic , holding)
raw_subtype	원본 데이터셋의 세부 유형
report_name	공시 보고서명
correction	정정 공시 여부
file_name	XML 파일명
document_role	주 문서 또는 첨부 문서 (MAIN , ATTACHMENT)
content_format	원문 형식. 이번 조사에서는 DART_XML
file_size_bytes	XML 파일 크기
relative_path	데이터셋 루트 기준 파일 상대 경로

XML 구조 정보

CSV 필드	의미
root_element_name	최상위 XML 태그 이름
document_name	첫 번째 DOCUMENT-NAME 태그의 정규화된 텍스트
max_depth	XML 태그의 최대 중첩 깊이
distinct_tag_count	서로 다른 태그 이름의 개수
total_element_count	시작 태그 전체 개수
section_1_count	SECTION-1 태그 개수
section_2_count	SECTION-2 태그 개수
section_3_count	SECTION-3 태그 개수
title_count	TITLE 태그 개수
paragraph_count	P 태그 개수
table_count	TABLE 태그 개수
table_row_count	표 행 태그 개수
table_header_count	표 헤더 셀 태그 개수
table_cell_count	표 일반 셀 태그 개수

보정, 성능 및 실패 정보

CSV 필드	의미
repaired_ampersand_count	읽는 동안 &amp; 로 보정한 단독 & 개수

CSV 필드	의미
repaired_less_than_count	읽는 동안 <로 보정한 비구조적 < 개수
elapsed_millis	파일 하나를 조사하는 데 걸린 시간
status	조사 성공 또는 실패 상태
error_type	실패 시 최하위 예외 클래스
error_line	XML 파서가 보고한 오류 행
error_column	XML 파서가 보고한 오류 열
error_message	최하위 예외 메시지

6. 페이지별 실행 결과

최대 배치 크기를 500으로 제한하고 페이지 번호를 바꾸어 전체 파일을 조사했다.

페이지	리포트 파일	대상 수	성공	실패	파일별 소요시간 합계
0	xml-structure-profile-page-000.csv	500	500	0	156,552ms
1	xml-structure-profile-page-001.csv	500	500	0	130,265ms
2	xml-structure-profile-page-002.csv	500	500	0	196,444ms
3	xml-structure-profile-page-003.csv	500	500	0	221,407ms
4	xml-structure-profile-page-004.csv	500	500	0	180,802ms
5	xml-structure-profile-page-005.csv	500	500	0	218,820ms
6	xml-structure-profile-page-006.csv	147	147	0	63,744ms
합계	7개 파일	3,147	3,147	0	1,168,034ms

파일별 소요시간 합계는 CSV의 elapsed_millis를 더한 값이다. 애플리케이션 시작, DB 조회, CSV 저장 시간까지 포함한 실제 전체 실행 시간과는 다를 수 있다.

초기 개발 과정에서 만든 다음 파일은 소규모 검증용 예비 리포트이므로 최종 전수 집계에서 제외했다.

- xml-structure-profile.csv
- xml-structure-profile-200.csv

7. 최종 조사 결과

완전성 검증

검증 항목	결과
DB의 DART XML 문서 수	3,147
최종 CSV 행 수	3,147

검증 항목	결과
고유 <code>disclosure_document_id</code> 수	3,147
중복 행 수	0
성공 수	3,147
실패 수	0
루트 태그가 <code>DOCUMENT</code> 인 문서	3,147

페이지 내부 ID가 오름차순이고, 다음 페이지의 시작 ID가 이전 페이지의 마지막 ID보다 큰 것도 확인했다. 마지막 페이지가 147건으로 끝났으므로 $500 \times 6 + 147 = 3,147$ 건 전체가 포함되었다.

공시 그룹별 분포

<code>source_group</code>	문서 수
<code>periodic</code>	1,466
<code>holding</code>	1,083
<code>major</code>	598
합계	3,147

문서 역할별 분포

<code>document_role</code>	문서 수
<code>MAIN</code>	2,732
<code>ATTACHMENT</code>	415
합계	3,147

`ATTACHMENT`도 별도의 XML 구조와 내용을 가질 수 있으므로, 실제 파서에서 주 문서와 첨부 문서를 구분하면서 둘 다 처리할 필요가 있다.

읽기 중 문자 보정 현황

보정 유형	보정이 발생한 파일 수	전체 보정 횟수
단독 <code>&</code>	1,620	84,075
비구조적 <code><</code> 및 본문 표현 태그	1,352	42,699

보정 건수가 많다는 것은 원본 데이터가 잘못되었다는 의미만은 아니다. DART 본문의 회사명, 상품명, 주식, 비교 표현 등에 XML 예약 문자인 `&`, `<`, `>`가 일반 텍스트처럼 포함된 사례가 많다는 의미다.

8. 성능 결과

항목	결과
전체 파일 크기 합계	5,531,987,496 bytes
평균 파일 크기	약 1,757,861 bytes
중앙 파일 크기	371,876 bytes
최대 파일 크기	32,199,881 bytes
파일당 평균 조사 시간	371.16ms
파일당 중앙 조사 시간	166ms
파일당 95백분위 조사 시간	1,284ms
가장 오래 걸린 파일	4,203ms

가장 오래 걸린 파일은 다음과 같다.

- 접수번호: 20250814004489
- 파일명: 20250814004489.xml
- 그룹: periodic
- 역할: MAIN
- 파일 크기: 20,368,056 bytes
- 조사 시간: 4,203ms

최종 결과 기준으로 큰 XML도 수 초 안에 처리되었으며, 일회성 내부 데이터 준비 프로그램의 목적에는 충분한 성능이다.

개발 중 중복 따옴표를 정규식으로 보정했던 버전에서는 특정 대형 파일 하나가 약 168초 걸렸다. 이를 문자 상태를 순차적으로 확인하는 상태 머신 방식으로 변경한 뒤 같은 계열 파일의 처리 시간이 수 초 수준으로 줄었다. 전체 파일을 메모리에 적재하지 않는 StAX 스트리밍 방식도 대형 파일 처리의 안정성을 높였다.

9. 발견한 비정상 XML 유형과 대응

실제 데이터에는 표준 XML 파서가 그대로 읽지 못하는 사례가 있었다. 문제를 숨기지 않고 각 실패의 원문 위치와 최하위 예외를 확인한 뒤, 확인된 유형만 읽기 단계에서 보정했다.

유형	원인 예시	대응
단독 &	회사명 또는 본문에 XML 엔티티가 아닌 & 사용	이미 유효한 엔티티가 아닌 경우에만 &로 변환
일반 비교 표현의 <	본문에서 수학·비교 기호로 < 사용	실제 태그 시작이 아닌 경우 <로 변환
속성값 내부 따옴표	ENG="Small office home office("SOHO")" 형태	상태 머신으로 속성의 시작·종료를 구분하고 내부 따옴표를 보정
속성 시작 직후 중복 따옴표	ENG=" "Snow Corporation" 형태	속성값 시작 직후의 중복 따옴표 제거

유형	원인 예시	대응
본문 표현을 태그처럼 작성	<CG>, <MANIFESTO>, <DataBada> 등 종료 태그 없는 표현	확인된 비구조적 이름만 일반 텍스트로 이스케이프

현재 확인된 비구조적 꺾쇠 표현 이름은 다음과 같다.

```
STS, CG, BGMI, DREAM, MANIFESTO,
DATABADA, GRANDATA, SIT, IIT, SHEESH
```

이 목록은 실제 실패 사례를 확인하면서 추가한 제한적 허용 목록이다. 임의의 모든 <문자열>을 텍스트로 바꾸지 않은 이유는 정상 XML 태그까지 손상시키지 않기 위해서다.

원본 보존 원칙

보정은 Reader가 XML을 읽어 파서에 전달하는 동안에만 적용된다.

- 원본 XML 파일은 수정하지 않는다.
- DB에 저장된 경로와 메타데이터도 수정하지 않는다.
- 조사 리포트에는 보정 횟수를 기록한다.
- 이후 실제 파서도 원문과 보정된 읽기 결과를 구분할 수 있어야 한다.

속성 따옴표 보정 횟수도 내부적으로 계산하지만, 현재 XmlStructureProfile과 CSV에는 해당 횟수 필드가 없다. 이후 보정 이력을 세밀하게 감사해야 한다면 repaired_attribute_quote_count를 결과 모델과 CSV에 추가해야 한다.

10. 파서 안전 설정

XML 파서는 외부 리소스를 읽지 못하도록 다음과 같이 설정했다.

- DTD 비활성화
- 외부 엔티티 비활성화
- 외부 파일 및 URL 해석 차단
- 네임스페이스 인식 활성화
- 연속된 문자 이벤트 병합 활성화

이는 공시 XML 안에 외부 파일 또는 URL 참조가 있어도 구조 조사 과정에서 외부 리소스에 접근하지 않도록 하기 위한 설정이다.

11. 실행 설정

구조 조사 실행 시 .env에서 페이지 번호, 페이지 크기, 출력 파일명을 지정했다.

```
FOLIOLENS_PROFILE_XML_BATCH_ON_STARTUP=true
FOLIOLENS_PROFILE_XML_BATCH_PAGE=0
FOLIOLENS_PROFILE_XML_BATCH_LIMIT=500
FOLIOLENS_PROFILE_XML_BATCH_REPORT_PATH=/reports/xml-structure-profile-page-000.csv
```

페이지를 실행할 때마다 **PAGE**와 **REPORT_PATH**의 페이지 번호를 함께 변경했다. 같은 출력 경로를 사용하면 기존 CSV가 덮어쓰이므로 페이지별로 다른 파일명을 사용했다.

전체 조사가 끝난 현재는 일반 애플리케이션 시작 시 다시 실행되지 않도록 다음처럼 비활성화한다.

```
FOLIOLENS_PROFILE_XML_BATCH_ON_STARTUP=false
```

12. 이번 조사로 확인한 파서 설계 방향

12.1 공통 진입점 사용 가능

3,147개 XML의 루트 태그가 모두 **DOCUMENT**였다. 따라서 실제 파서는 **DOCUMENT**를 공통 진입점으로 사용할 수 있다.

12.2 계층 구조를 유지해야 함

공시 원문은 **SECTION-1**, **SECTION-2**, **SECTION-3** 같은 장-절 계층을 사용한다. 본문 검색 결과에서 원문 근거 위치를 보여주려면 파싱할 때 다음 정보를 함께 보존해야 한다.

- 상위 섹션 경로
- 문서 안에서의 등장 순서
- 제목
- 문단 또는 표 여부
- 원본 파일 식별자
- 가능한 경우 원문 행 번호

12.3 표를 별도로 처리해야 함

재무 수치와 핵심 공시 정보는 표에 포함되는 경우가 많다. **TABLE**, 행, 헤더 셀, 일반 셀을 단순 텍스트로 합치면 행과 열의 관계가 사라진다. 실제 파서에서는 표의 행-열 구조를 유지한 결과 모델이 필요하다.

12.4 파일 유형별 차이를 고려해야 함

조사 대상에는 **periodic**, **holding**, **major** 그룹과 **MAIN**, **ATTACHMENT** 역할이 모두 존재한다. 하나의 대표 사업보고서만 기준으로 파서를 만들지 않고, 그룹과 역할별 대표 문서를 자동화 테스트에 포함해야 한다.

12.5 같은 보정기를 재사용해야 함

구조 조사에서 성공한 XML 읽기 규칙과 실제 원문 파서의 읽기 규칙이 다르면 다시 같은 실패가 발생한다. 현재 **XmlStructureProfiler** 내부의 보정 Reader를 독립 구성요소로 분리해 실제 파서와 공유하는 것이 적절하다.

13. 이번 단계에서 하지 않은 일

이번 작업은 구조 조사 단계이므로 다음 작업은 아직 수행하지 않았다.

- 장·절·문단·표 본문을 DB에 저장
- 공시 핵심 사실 추출
- 금액, 날짜, 비율 계산
- 검색용 청크 생성
- 임베딩 또는 전문 검색 인덱스 생성
- 원문 근거 위치를 API로 제공
- HTML 및 PDF 원문 파싱
- LLM 분석 또는 답변 생성

즉, **XML을 안정적으로 읽을 수 있다는 것을 전수 검증한 상태**이며, 서비스에서 사용할 구조화 데이터 적재는 다음 단계다.

14. 현재 방식의 한계

- CSV에는 태그별 전체 맵이 아니라 주요 태그의 집계값만 저장한다.
- 본문 내용과 표의 실제 값은 저장하지 않는다.
- 비구조적 꺾쇠 표현은 전수 데이터에서 발견한 이름 목록에 의존한다.
- 속성 따옴표 보정 횟수는 아직 CSV에 노출되지 않는다.
- 페이지 순서는 접수일 순서가 아니라 `disclosure_document_id` 오름차순이다.
- 이번 성공 결과는 DART XML에만 해당하며 HTML과 PDF의 파싱 가능성을 보장하지 않는다.

15. 관련 파일

구현 코드

```
backend/src/main/java/com/foliolens/backend/disclosure/infrastructure/profiling/
├─ XmlStructureProfile.java
├─ XmlStructureProfiler.java
├─ XmlStructureProfileStatus.java
├─ XmlStructureProfileRow.java
├─ XmlStructureProfileBatchResult.java
├─ XmlStructureProfileBatchService.java
├─ XmlStructureProfileReportWriter.java
└─ XmlStructureProfileBatchRunner.java
```

```
backend/src/main/java/com/foliolens/backend/disclosure/infrastructure/persistence/
└─ DisclosureDocumentRepository.java
```

```
backend/src/test/java/com/foliolens/backend/disclosure/infrastructure/profiling/
└─ XmlStructureProfilerTest.java
```

최종 리포트

```
reports/
├─ xml-structure-profile-page-000.csv
├─ xml-structure-profile-page-001.csv
├─ xml-structure-profile-page-002.csv
├─ xml-structure-profile-page-003.csv
├─ xml-structure-profile-page-004.csv
├─ xml-structure-profile-page-005.csv
└─ xml-structure-profile-page-006.csv
```

16. 다음 작업

구조 조사가 완료되었으므로 다음 순서로 실제 XML 원문 파싱을 진행한다.

1. `XmlStructureProfiler` 내부의 보정 Reader를 `DartXmlSanitizingReader`와 같은 독립 구성요소로 분리한다.
2. 파싱 결과 모델을 정의한다.
 - `ParsedDisclosureDocument`
 - `ParsedDisclosureSection`
 - `ParsedDisclosureBlock`
 - 표, 행, 셀 모델
3. StAX 기반 `DartXmlDisclosureParser`를 구현한다.
4. 섹션 경로, 본문 순서, 제목, 문단, 표 구조, 원문 위치를 보존한다.
5. `major`, `periodic`, `holding`, `MAIN`, `ATTACHMENT` 대표 파일로 자동화 테스트를 작성한다.
6. 파싱 결과를 확인한 후 `disclosure_sections`, 본문 블록 또는 검색 청크 테이블을 확정한다.
7. 소규모 적재 후 검증하고 전체 3,147개 XML을 배치 적재한다.
8. 구조화된 원문을 검색·근거 제시·계산 로직에서 사용한다.

17. 완료 체크리스트

- ☒ 단일 XML 구조 조사기 구현
- ☒ 실패 원인의 최하위 예외와 행·열 기록
- ☒ 비정상 XML 문자 및 속성 따옴표 대응
- ☒ 대형 파일 처리 성능 개선
- ☒ 페이지 단위 배치 조사 구현
- ☒ 전체 DART XML 3,147개 조사
- ☒ 중복 및 누락 검증
- ☒ 최종 실패 0건 확인
- ☒ 구조 조사 결과 문서화
- ☐ 실제 XML 본문 파서 구현
- ☐ 구조화 결과 DB 적재
- ☐ 검색 및 근거 연결 구현